

ULTRON

Freebuff-Standards Coding Agent Upgrade Plan

An implementation blueprint for evolving the existing Ultron codebase into a reliable, autonomous, local-first coding agent - without rewriting the foundations that already work.

Document	Ultron Coding Agent Upgrade Specification
Scope	Coding agent only - no phone, voice, or general device control yet
Inference	Local llama.cpp / llama-server foundation
Primary objective	Freebuff-level coding workflow with reliable delegation, context management, verification, and recovery
Validation rule	Unit/integration tests + real model-in-the-loop Ultron tasks

SOURCE-OF-TRUTH RULE

The coding agent must inspect the actual repository state before making implementation decisions. This document defines the target architecture and progression; it does not authorize inventing files, classes, APIs, or behavior that are not present in the current codebase.

Prepared for Project Ultron

Baseline checked against the current Ultron main branch and the available project snapshot.

1. Executive Summary

The goal is not to copy Freebuff line-for-line. The goal is to make Ultron behave like a serious autonomous coding agent while preserving Ultron's existing architecture: the ReAct reasoning loop, canonical tool registry, capability routing, SecurityBoundary, task state, orchestration/delegation primitives, artifact model, and local llama.cpp engine.

The central architectural change is the introduction of a single **AgentRuntime**. Today, Ultron has strong live-agent primitives and a separate orchestration/delegation stack. The runtime will unify those paths so that delegation, context, verification, cancellation, budgets, and artifacts become part of the same coding execution lifecycle.

Target outcome

- **Autonomous repository work:** inspect, plan, edit, test, diagnose, repair, review, and report.
- **Selective delegation:** the root ReAct agent can delegate complex work to deterministic specialist agents through the existing Supervisor.
- **Repository-aware context:** retrieve only the files, symbols, history, and artifacts relevant to the current task.
- **Hard verification:** completion requires evidence, not an LLM completion claim.
- **Safe execution:** all tool actions remain mediated by the SecurityBoundary.
- **Local-first inference:** the agent remains model-provider agnostic and continues using the LlamaCppEngine/llama-server path.
- **Debuggable runs:** every task has state, events, artifacts, delegation lineage, tool outcomes, and failure information.

Non-goals for this upgrade

- Phone companion app, voice interface, mobile app control, general desktop assistant behavior.
- Replacing llama.cpp or introducing a new model provider.
- Replacing the existing ReActAgent with a completely different agent framework.
- Uncontrolled multi-agent swarms for every request.
- Giving the LLM direct unrestricted OS access.

2. Current Ultron Baseline

The current repository already contains several pieces required for this upgrade. The correct strategy is consolidation and integration, not a green-field rewrite.

Existing capability	Current role	Upgrade treatment
ReActAgent	Live model-driven reasoning/action loop	Keep; make it the root reasoning policy inside AgentRuntime
BaseEngine	Provider-neutral inference contract	Keep unchanged at architectural boundary
LlamaCppEngine	OpenAI-compatible local llama-server client	Keep; remain below agent runtime
Canonical tool registry	Tool discovery/schema/execution surface	Keep as the single tool source
Capability routing	Intent/capability/tool routing	Keep; use for deterministic routing
SecurityBoundary	Allow/confirm/deny and action controls	Keep as authoritative execution gate
TaskState / planning	Task lifecycle and plan tracking	Promote into runtime-owned state
Supervisor / DelegationRequest	Sequential specialist delegation	Connect directly to root coding loop
AgentRegistry	Specialist definitions and scoped execution	Use as delegation target registry
AgentArtifact / ArtifactStore	Structured cross-agent channel	Use for delegation results and provenance
Parallel read-only tools	Batching safe independent reads	Retain; make runtime-aware

Important baseline finding

The repository already has a meaningful delegation implementation. The Supervisor is intentionally deterministic, sequential, scoped by permissions, budget, and timeout, and passes structured artifacts rather than raw agent trajectories. That is a strong foundation. The upgrade should connect this machinery to the live ReAct coding path rather than create a second delegation framework.

Repository evidence

The current delegation implementation documents the lifecycle as TaskState -> Supervisor -> DelegationRequest -> specialist Agent -> AgentResult/artifact -> TaskState update. It also defines frozen permission profiles, execution budgets, timeouts, and lifecycle states such as PENDING, ASSIGNED, RUNNING, WAITING, COMPLETED, FAILED, BLOCKED, and CANCELLED.

Repository reference: github.com/Aravindhan450/ultron

3. Current Architecture vs Target Architecture

Current live execution path

```
CLI / user request
  |
  v
ReActAgent
  |
  v
Tool Registry
  |
  v
SecurityBoundary
  |
  v
Tools
  |
  v
BaseEngine -> LlamaCppEngine -> llama-server
```

Current orchestration path (partly separate)

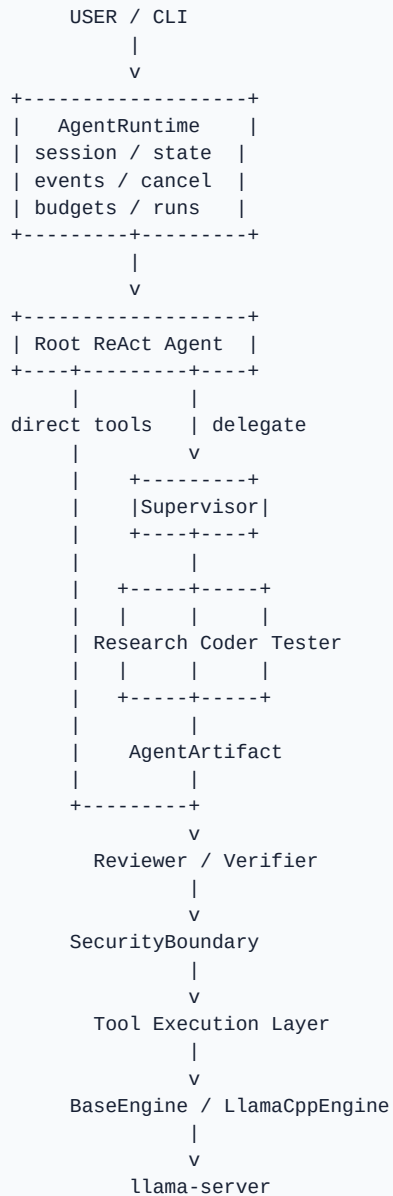
```
TaskState / Planning
  |
  v
Supervisor -> AgentRegistry -> Specialist
  |               |
  +---- AgentArtifact <-----+
  |
  v
orchestration result
```

Target unified path

```
CLI -> AgentRuntime -> Root ReAct Agent
                        |
                        | +--> direct tool execution
                        |
                        +--> Planner / Context Manager
                        |
                        +--> Supervisor -> Specialist Agents
                                |
                                v
                                AgentArtifact
                                |
                                v
                                Root Agent
                                |
                                v
                                verify -> review -> complete
                                |
                                v
                                SecurityBoundary -> Tool -> Engine
```

4. Designed Target Architecture

This is the architecture the coding agent should keep visible while implementing the upgrade.



5. AgentRuntime - The Architectural Hinge

AgentRuntime is the most important new component. It should become the owner of a coding run without becoming the reasoning model itself.

Responsibilities

- Create and own a run/session identifier.
- Initialize TaskState and execution budgets.
- Build the initial context through ContextManager.
- Invoke the Root ReAct Agent.
- Route direct tool calls through the canonical tool path.
- Invoke Supervisor for delegated work.
- Track tool, delegation, verification, and review events.
- Support cancellation and future steering/interruption.
- Handle bounded retries and recovery.
- Determine whether the task has enough evidence to be marked complete.
- Persist structured run outcomes and artifacts.

Suggested conceptual package

```
core/runtime/  
- runtime.py  
- session.py  
- context.py  
- events.py  
- cancellation.py  
- run_state.py
```

Critical separation

Component	Owns
AgentRuntime	Lifecycle, execution orchestration, state, cancellation, budgets, verification policy
ReActAgent	Reasoning loop and action selection
Supervisor	Deterministic delegation routing and specialist lifecycle
ContextManager	What information is assembled for a model invocation
Tool layer	Actual operations
SecurityBoundary	Whether an operation is allowed
Engine	How model inference happens

6. End-to-End Coding Workflow

The coding agent should follow this lifecycle for non-trivial repository work.

```
1. Receive request
  |
2. Create AgentRuntime session + TaskState
  |
3. Discover repository structure
  |
4. Retrieve relevant files / symbols / history
  |
5. Plan task + define verification criteria
  |
6. Root ReAct reasoning loop
  |
  +--> direct read/search/tool calls
  |
  +--> delegate complex subtask
        |
        v
      specialist executes
        |
        v
      structured artifact
        |
        +----> root agent
  |
7. Apply changes through SecurityBoundary
  |
8. Run tests / typechecks / targeted checks
  |
9. Diagnose failures and repair
  |
10. Review actual diff + verification evidence
  |
11. PASS? ---- NO ----> repair loop
    |
    YES
    |
12. Record completion evidence + report
```

The central principle is that reasoning is not completion. A model can propose a correct-looking patch and still fail the user's goal. Ultron must verify the repository state after its own modifications.

7. Freebuff-Style Delegation Model

Delegation is a first-class path from the root coding agent to specialized agents. It should be selective, deterministic, bounded, and artifact-driven.



Delegation rules

- **Do not delegate trivial tasks.** Simple read/edit/test work should remain in the root loop.
- **Delegate bounded objectives.** A specialist receives one concrete objective and explicit constraints.
- **No raw reasoning sharing.** Cross-agent communication is structured artifacts/results only.
- **Permissions are frozen.** A specialist cannot widen its own permission profile.
- **Budgets are bounded.** Every delegated run has a timeout and execution budget.
- **Sequential first.** Keep the existing sequential Supervisor until the sequential path is proven reliable.
- **Artifacts are the contract.** The root agent consumes findings/results, not internal trajectories.

Recommended specialist roles

Agent	Use when	Output
Researcher	External docs, unfamiliar framework, broad investigation	ResearchFinding
Coder	Large or multi-file implementation	ImplementationResult
Tester	Complex verification strategy	TestResult
Reviewer	Final diff and regression review	ReviewResult

8. Repository Context Engine

A Freebuff-level coding agent cannot repeatedly dump entire repositories into the context window. Context must be assembled deliberately.

```
Repository
|
+--> directory / file inventory
+--> code / symbol map
+--> lexical search
+--> semantic retrieval (where justified)
+--> git state / diff / recent changes
+--> task history / artifacts
    |
    v
  ContextManager
    |
    +--> relevance filtering
    +--> deduplication
    +--> token budgeting
    +--> compaction
    +--> important-fact retention
    |
    v
  model context
```

ContextManager responsibilities

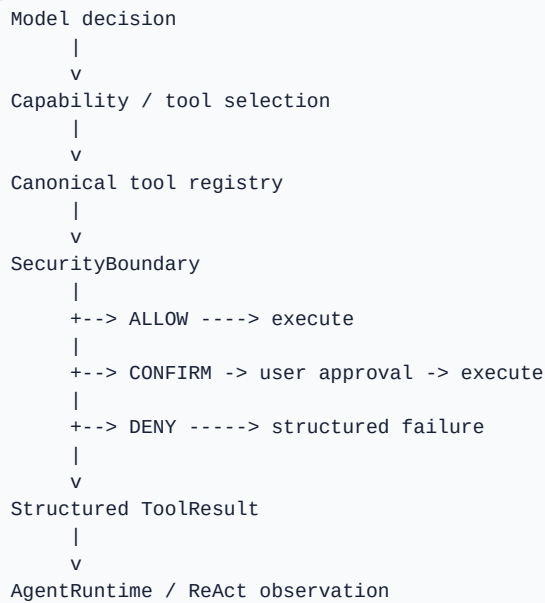
- Start with repository structure, not arbitrary file dumps.
- Retrieve only files relevant to the current objective.
- Prefer targeted windows around symbols/lines over full large files.
- Keep the current plan and verification criteria visible.
- Summarize stale tool observations instead of retaining every raw output.
- Preserve critical errors, changed files, test results, and user constraints.
- Avoid duplicate context when the same content is retrieved multiple times.

Anti-hallucination context rule

If the agent cannot locate a file, class, function, API, table, command, or configuration entry in the current repository, it must not assume that it exists. It must search again, inspect adjacent architecture, or explicitly report uncertainty.

9. Tool and Security Architecture

The model should never receive a direct OS execution primitive. Every action follows the existing mediated tool path.



Coding tools to prioritize

Category	Core operations
Discovery	list_directory, glob, code_search
Reading	read_files / targeted reads
Editing	str_replace, write_file / patch-oriented editing
Terminal	run_terminal_command
Verification	tests, typecheck, lint, build where available
Git awareness	status, diff, changed-file inspection
Future	browser / external research tools when required

Security invariant

All direct and delegated agents must converge on the same SecurityBoundary. Delegation must not create a side door around path restrictions, command restrictions, confirmation requirements, or audit logging.

10. Verification and Completion Gate

This is where the agent becomes reliable instead of merely impressive.

Two-layer validation is mandatory

Layer	What it proves	Examples
Automated	Contracts and regressions are correct	unit tests, integration tests, lifecycle tests, tool tests, security tests
Model-in-the-loop	The real Ultron path can solve user tasks	CLI task -> model -> tools -> edits -> tests -> final result

Completion contract

```
Task complete candidate
|
+--> required files changed?
+--> expected behavior implemented?
+--> relevant tests executed?
+--> tests / checks pass?
+--> actual diff reviewed?
+--> no unresolved critical errors?
|
v
YES
|
v
Runtime marks task COMPLETE
|
v
Final response includes evidence
```

The model's statement 'done' is never sufficient evidence.

Failure recovery

- Capture the failing command/output as structured evidence.
- Associate the failure with the relevant task step.
- Refresh context around the failure.
- Let the root agent diagnose and repair.
- Re-run the smallest relevant verification first, then broader checks.
- Stop after bounded retries and report the blocker instead of looping indefinitely.

11. Complete Upgrade Roadmap

Implement in this order. Do not jump ahead because a later feature looks more exciting.

Phase	Goal	Primary deliverable	Exit condition
0	Baseline	Repository inspection + architecture map	Current live paths and tests documented
1	AgentRuntime	Unified runtime around existing ReActAgent	All existing coding behavior still works through Runtime
2	Context Engine	Repository-aware context retrieval/compaction	Model receives targeted context on long tasks
3	Coding Loop	Edit -> verify -> repair loop	Real coding tasks recover from failures
4	Delegation	Root -> Supervisor -> specialist -> artifact -> root	Real task successfully uses delegation
5	Reviewer	Diff/test evidence review gate	False completion is rejected
6	Observability	Events, budgets, cancellation, run trace	Failed runs are diagnosable and bounded

Phase 0 is mandatory before implementation

- 1 Inspect the current main branch and working tree.
- 2 Identify the exact current entryptoint for coding requests.
- 3 Map current ReActAgent, tool registry, SecurityBoundary, TaskState, planning, Supervisor, AgentRegistry, ArtifactStore, and engine integration.
- 4 Read relevant tests before modifying behavior.
- 5 Record discrepancies between this document and the actual repository.
- 6 Do not create compatibility shims for architecture that does not exist.

12. Phase 1 Implementation Specification - AgentRuntime

This is the first coding milestone and should be implemented before major delegation/context changes.

Suggested interfaces

```
AgentRuntime.run(request) -> RunResult
AgentRuntime.cancel(run_id) -> CancellationResult
AgentRuntime.get_status(run_id) -> RunStatus
AgentRuntime.execute_tool(...) -> ToolResult
AgentRuntime.delegate(...) -> AgentResult / Artifact
AgentRuntime.verify(...) -> VerificationResult
```

Run state should track at minimum

- run_id / task_id
- current lifecycle state
- current plan step
- iteration count
- tool call count
- delegation count
- execution budget usage
- changed files
- verification evidence
- errors / recovery attempts
- artifacts produced
- timestamps

Implementation constraints

- Do not duplicate AgentStatus or create a second incompatible lifecycle state machine.
- Reuse existing TaskState and AgentStatus concepts where appropriate.
- Do not move SecurityBoundary responsibility into AgentRuntime; Runtime orchestrates, SecurityBoundary authorizes.
- Do not put provider-specific llama.cpp logic into AgentRuntime.
- Do not make Supervisor LLM-driven.
- Preserve current ReAct tool schema and tool registry contracts unless tests prove they must change.
- Keep public APIs backwards-compatible where practical; if a breaking change is necessary, update all call sites and tests in the same change.

13. Phase 2-3 - Context and Autonomous Coding Loop

After Runtime is stable, the next objective is to make Ultron genuinely good at repository work.

Repository discovery strategy

- 1 Identify repository root and relevant project boundaries.
- 2 Inspect top-level structure and project metadata.
- 3 Detect language/build/test tooling from actual files.
- 4 Search for the user's target symbol, feature, error, or behavior.
- 5 Read only the relevant files and adjacent interfaces.
- 6 Trace call sites before modifying shared functions.
- 7 Inspect existing tests and conventions before creating new ones.

Editing strategy

- Prefer small, explicit patches over rewriting whole files.
- Preserve existing architecture and naming unless the task explicitly requires redesign.
- After each logical mutation, inspect the resulting diff.
- Never claim an edit succeeded without verifying the file state.
- Never fabricate an API or import because it 'should' exist.

Verification strategy

```
change
|
+--> syntax / import check
+--> targeted unit test
+--> related integration test
+--> typecheck / lint
+--> broader suite when justified
|
v
failure?
| yes -> diagnose -> repair -> repeat
| no
v
review actual diff
|
v
record evidence
```

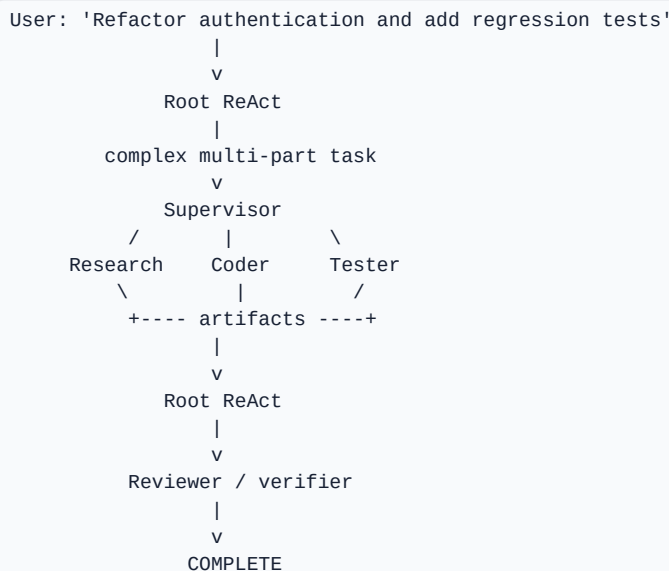
14. Phase 4-5 - Delegation and Review

The existing Supervisor/DelegationRequest implementation should be wired into the live runtime rather than replaced.

Integration sequence

- 1 Expose delegation as a controlled capability available to the root agent.
- 2 Root agent produces a bounded delegation objective.
- 3 Runtime asks Supervisor to construct and execute the delegation.
- 4 Supervisor selects an AgentSpec from AgentRegistry.
- 5 Specialist runs in an ExecutionContext with the frozen permission profile.
- 6 Specialist returns AgentResult plus structured AgentArtifact.
- 7 Runtime records delegation outcome and feeds the artifact back to the root task.
- 8 Root agent continues from the artifact; it does not receive the specialist's hidden trajectory.
- 9 Reviewer validates the resulting implementation and evidence.

Example



Do not introduce parallel delegation until sequential delegation has stable tests and model-in-the-loop evidence.

15. Anti-Hallucination Engineering Rules

These rules are intended to be included in the coding agent's working instructions and used during every upgrade task.

Rule	Required behavior
Inspect before modifying	Read the actual file, callers, tests, and interfaces before proposing changes.
No invented architecture	If a referenced class/module/API is absent, search the repository before assuming it exists.
Evidence over assumption	Use actual tool output, diffs, tests, and file contents as ground truth.
One source of truth	Do not create duplicate registries, lifecycle systems, tool paths, or security gates.
Minimal change	Prefer the smallest architecture-preserving implementation that satisfies the phase.
Trace dependencies	Before changing shared contracts, find all call sites and tests.
Test real behavior	Passing unit tests is necessary but not enough; run model-in-the-loop tasks.
No false completion	Never mark a task complete because the model generated a plausible response.
Explicit uncertainty	If repository state cannot be verified, stop and report what is unknown.
Bounded autonomy	Every loop, delegation, retry, and command execution has limits.
Security is non-bypassable	All agents use the same SecurityBoundary.

16. Instructions for the AI Coding Agent

The following can be used as the persistent implementation brief for the coding agent working on this upgrade.

PROJECT: Ultron

OBJECTIVE: Upgrade Ultron into a Freebuff-standard autonomous coding agent while preserving its existing foundations.

WORKING RULE: Before changing code, inspect the current repository and verify that every referenced file/class/function actually exists. Treat the repository as the source of truth and this document as the target architecture/roadmap.

DO NOT: rewrite working subsystems without evidence; invent APIs; create duplicate registries/lifecycle systems/tool paths; bypass SecurityBoundary; replace ReActAgent unnecessarily; make Supervisor an LLM; or mark work complete without verification.

IMPLEMENTATION ORDER: baseline inspection -> AgentRuntime -> ContextManager -> autonomous edit/verify/repair loop -> live delegation -> reviewer -> observability/recovery.

VALIDATION: every phase requires automated tests plus real model-in-the-loop Ultron tasks. Report both results.

DELEGATION: Root ReAct -> deterministic Supervisor -> specialist -> structured AgentArtifact -> Root ReAct. Preserve bounded permissions, budgets, timeout, lifecycle, and artifact-only cross-agent communication.

COMPLETION: A task is complete only when the requested behavior is implemented, relevant checks have been executed, evidence is available, the actual diff is acceptable, and no critical unresolved failure remains.

Recommended coding-agent loop

```
READ THIS DOCUMENT
  |
  v
INSPECT CURRENT REPOSITORY
  |
  v
MAP CURRENT IMPLEMENTATION
  |
  v
COMPARE CURRENT vs TARGET
  |
  v
IMPLEMENT ONE PHASE
  |
  v
RUN AUTOMATED TESTS
  |
  v
RUN REAL ULTRON MODEL TEST
  |
  v
REPORT EVIDENCE + UPDATE STATUS
  |
  v
ONLY THEN MOVE TO NEXT PHASE
```

17. Definition of Done - Freebuff-Level Coding Agent

Ultron should not be considered upgraded merely because the new classes exist. The system is done when the behavior is demonstrably present.

Capability	Definition of done
Runtime	All coding runs are owned by AgentRuntime with bounded lifecycle, state, events, cancellation, and budgets.
Reasoning	Root ReActAgent can inspect, act, observe, and recover without losing task state.
Context	Long coding tasks use targeted repository context and compaction rather than uncontrolled transcript growth.
Editing	Agent can make multi-file changes safely and inspect its own diff.
Verification	Relevant tests/checks run automatically for non-trivial changes.
Recovery	Tool/test failures trigger bounded diagnosis and repair loops.
Delegation	Root agent can delegate bounded work through Supervisor and consume structured artifacts.
Security	No direct agent path bypasses SecurityBoundary.
Review	Actual changes and verification evidence are reviewed before completion.
Observability	A failed task can be reconstructed from structured run/task/tool/delegation records.
Model-in-loop	Representative real coding tasks succeed through the full CLI -> runtime -> model -> tools -> verification path.

Representative acceptance tasks

- Fix a small bug in an existing repository without delegation.
- Implement a multi-file feature and automatically run its tests.
- Introduce an intentional failing change and verify that Ultron detects and repairs it.
- Perform a complex task that causes the root agent to delegate research or testing.
- Run a long task requiring context compaction without losing critical requirements.
- Attempt a disallowed operation and verify SecurityBoundary blocks it.
- Cancel an active run and verify the runtime reaches a clean terminal state.

18. Risks and Guardrails

Risk	Why it happens	Guardrail
Runtime becomes a god object	Too many responsibilities migrate into one class	Runtime owns lifecycle; delegate context, security, planning, tools, and inference to specialized components.
Agent swarm explosion	Delegation used for every task	Selective delegation; sequential first; budgets/timeouts.
Context bloat	Raw tool output/transcripts retained	ContextManager compaction and artifact summaries.
False completion	Model says task is done	Hard verification + diff review.
Security bypass	Specialist gets a direct executor	All agents converge on canonical tools + SecurityBoundary.
Duplicate architecture	New system created beside old one	One registry, one lifecycle model, one tool path, one artifact channel.
Local inference latency	Too many model calls	Prefer root ReAct for simple tasks; delegate only when complexity warrants it.
Infinite recovery	Repeated failures trigger endless retries	Bounded retries, budgets, timeouts, terminal failure state.
Repository hallucination	Agent assumes files/APIs exist	Inspect/search first; uncertainty must be explicit.

19. Implementation Checklist

Use this as the progress checklist while upgrading Ultron.

Status	Item
☐	Inspect current main branch and working tree before each phase
☐	Document current live coding endpoint
☐	Introduce AgentRuntime without changing existing behavior
☐	Route CLI coding execution through AgentRuntime
☐	Add runtime lifecycle/state/events/cancellation/budget tracking
☐	Build ContextManager and repository-aware retrieval
☐	Add context compaction and relevance controls
☐	Harden edit -> verify -> repair loop
☐	Integrate Supervisor delegation into live ReAct path
☐	Validate specialist permissions/budgets/timeouts
☐	Use AgentArtifact as the delegation result contract
☐	Add reviewer/diff verification gate
☐	Add run observability and failure recovery
☐	Pass automated test suite
☐	Pass model-in-the-loop coding acceptance tasks
☐	Only then declare Freebuff-level coding-agent milestone complete

Final architectural principle

Build Ultron as a single coherent coding runtime, not a collection of impressive agent features. The root ReAct agent reasons, the runtime orchestrates, the context manager supplies evidence, the Supervisor delegates, specialists produce artifacts, the SecurityBoundary controls execution, tools perform work, and verification decides whether the user's goal was actually achieved.

End of specification.